From TE-CCL to Graph Neural Policy Learning for Adaptive Collective Communication

POLITECNICO
MILANO 1863

Problem: Optimizing collective communication over heterogeneous accelerator topologies — especially in multi-chassis or cross-region-like settings— is computationally expensive.

Current solution: TE-CCL uses MILP optimization to generate high-quality schedules, but solving can be computationally expensive.

Our goal: Learn this optimization behavior with graph neural networks in a using a "Teacher-Student" framework , so that high-quality communication decisions can be produced much faster

Pipeline Overview

Step 1 — Teacher data extraction

TE-CCL outputs are parsed into:

- topology graphs
- policy matrices
- collective finish times

Step 2 — Agent

Input: topology graph

Output: edge-level communication policy

Step 3 — SimNet

Input: topology graph + policy

Output: predicted collective completion time

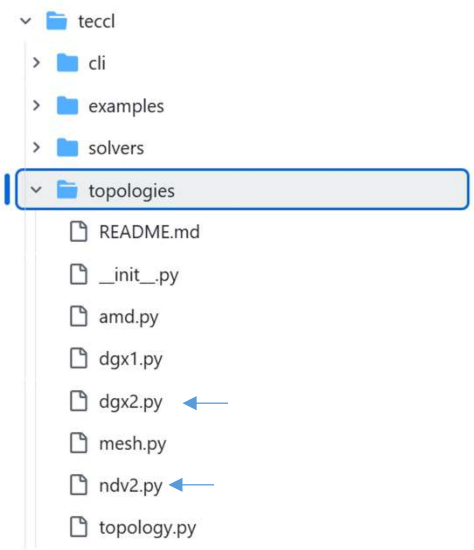
End-to-end goal

Topology → Agent → Policy → SimNet → Time

TE-CCL Repository as Teacher and Data Source

2 components of the TE-CCL repository were used:

1. Topologies : To construct communication graphs



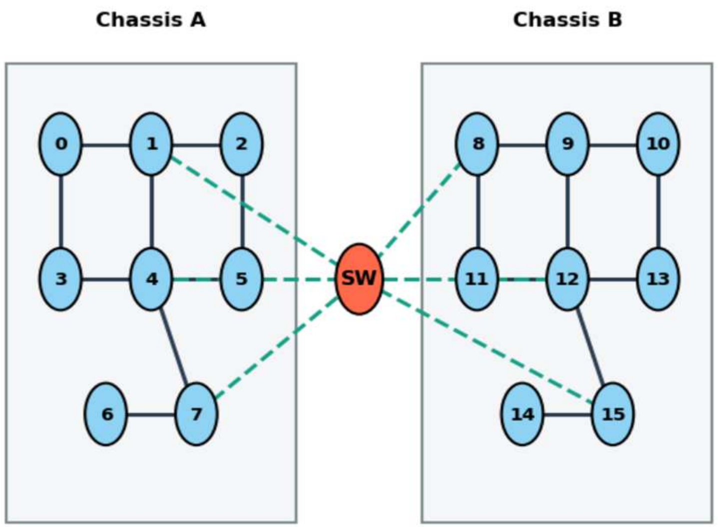
- NDv2 and DGX2 were the two topology families used in the final dataset, since they are experiments with them provided with the TE-CCL expert outputs.

- Topologies were defined with :
  Capacity → link bandwidth
  Alpha → link latency
  Switch indices → switch node identities
  Chassis → multi-chassis structure
  Nodes per chassis → local topology size

Topologies Used in Final Dataset

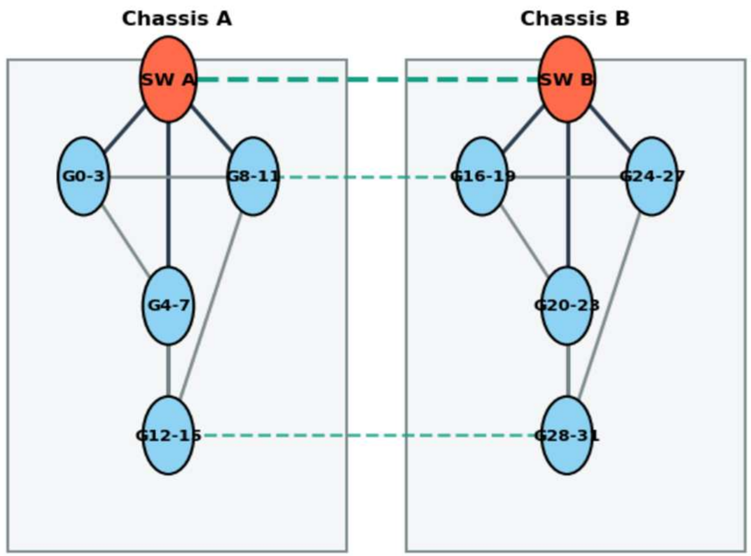
POLITECNICO
MILANO 1863

NDv2 Topology Abstraction



Compact 2-chassis structure
8 GPUs per chassis

DGX2 Topology Abstraction



Larger, switch-based hierarchy
16 GPUs per chassis

Schematic view: node positions are illustrative; dashed links indicate cross-chassis communication.

Schematic abstraction: GPU groups are aggregated for readability; dashed links indicate cross-chassis interconnects.

TE-CCL Repository as Teacher and Data Source

POLITECNICO MILANO 1863

2. Experiments with Provided outputs : To extract expert schedules and collective completion times as supervision targets

Each output file represents a combination of configurations:

Component	Final Dataset
Topologies	NDv2, DGX2
Chassis	2, 4 (4-chassis mainly from NDv2)
Collectives	AllGather, AlltoAll
Modes	Fast, Slow, Fast_Early_Stop
Message sizes	1KB – 1GB
Total samples	154

Graph Construction from Topology

Each topology is converted into a directed graph :

An $\boldsymbol{edge_{ij}} \in \boldsymbol{E}$ exists if,

$$\{\boldsymbol{capacity}\}_{\{ij\}} > \mathbf{0}$$

Each edge is represented by:

$$\boldsymbol{edge_{ij}} = [\text{capacity}_{ij},\ \boldsymbol{\alpha_{ij}},\ \text{switch}_{ij}]$$

where,

$$\boldsymbol{\alpha_{ij}}\text{: link latency} \qquad \text{switch}_{ij} \in \{\mathbf{0}, \mathbf{1}\}\text{: switch involvement}$$

So, the graph is fully defined by:

- nodes (GPUs and switches)
- edges (communication links)
- edge attributes (bandwidth, latency, switch indicator)

Policy Matrix Extraction from TE-CCL Flows

Each TE-CCL output provides a set of communication flows of the form:

Chunk $\boldsymbol{c}$ traveled over $\boldsymbol{i} \rightarrow \boldsymbol{j}$

The policy matrix $\boldsymbol{P} \in \boldsymbol{R}^{\boldsymbol{N} \times \boldsymbol{N}}$ constructed as :

$$\boldsymbol{P}_{\boldsymbol{ij}} = \sum_{\text{flows}} \mathbf{1}\,\{\text{edge } \boldsymbol{i} \rightarrow \boldsymbol{j} \text{ is used}\}$$

Then normalized,

$$\widehat{\boldsymbol{P}_{\boldsymbol{ij}}} = \frac{\boldsymbol{P}_{\boldsymbol{ij}}}{\max_{\boldsymbol{a},\boldsymbol{b}} \boldsymbol{P}_{\boldsymbol{ab}}} \qquad \widehat{\boldsymbol{P}_{\boldsymbol{ij}}} \in [\mathbf{0}, \mathbf{1}]$$

The Agent GNN

Input: Graph

↓

3 layer GNN Encoder (GATv2)
The GNN encoder updates each node by looking at its neighbors and the quality of connections.

↓

Node Embeddings
This produces node embeddings that capture the structure of the network.

↓

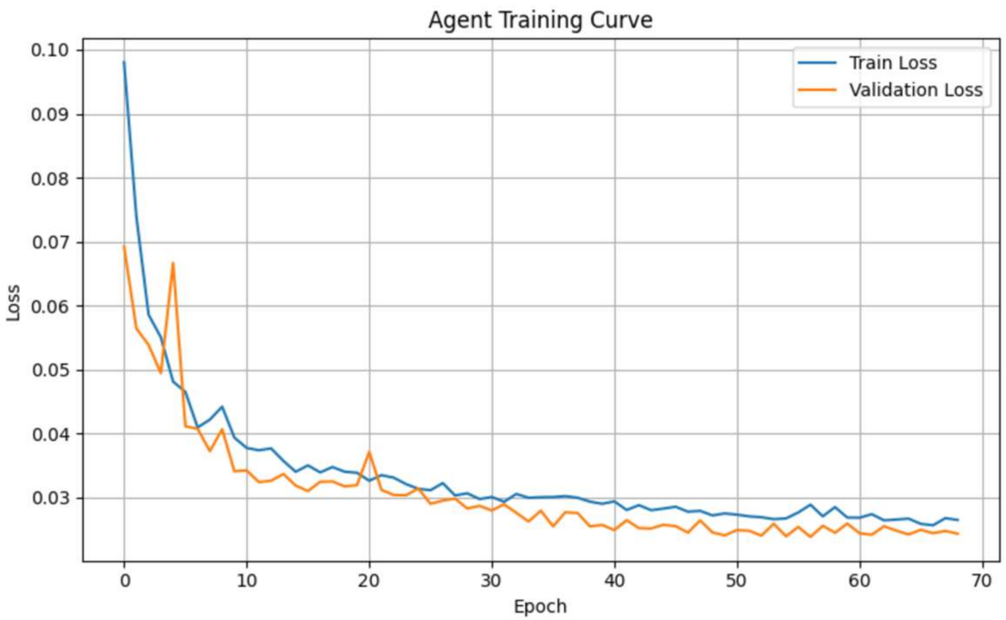
Edge Scoring (MLP)
Then, for each edge, we combine the two endpoint embeddings and edge features to compute a score.

↓

Output: Policy Matrix
Finally, we obtain a policy score for every communication link.

GATv2 is preffered as an attention mechanism and it allows the model to weight neighbors differently based on the varying edge attributes.

Training Results



Best validation loss: 0.0237
Test loss: 0.0228

- Early stopping applied.
- The training and validation losses decrease consistently, indicating stable learning.
- The gap between them remains small, suggesting good generalization.
- The model alone successfully learns to approximate the expert communication policies.

SimNet : The Proxy Model